MODULE 22 · DOMAIN 8 - HCP TERRAFORM

HCP Terraform: Workspaces, Projects, Variable Sets

The unit of organisation, the way to group it, and how values reach a run.

DOMAIN 8 of 8 COURSE WEIGHT 12% MODULE 22 of 27

admin@sthithapragna.com

A workspace is the unit of everything. A project is how you avoid managing two hundred of them one at a time.

What an HCP Terraform workspace holds

- **One Terraform configuration**, from a repository, an upload, or the CLI.
- **One state file**, versioned, with every previous version retained.
- **Its own variable values**, including secrets, stored by the service.
- **Its own run history**: every plan and apply, who started it, and who approved it.
- **Its own permissions**, so who can plan and who can apply are separate questions.

> ON THE EXAM
>
> Module eighteen's contrast: a CLI workspace is a state file, and an HCP workspace is all of this. Learn both definitions.

Workspace settings

- **Execution mode**: Remote, Local or Agent. Module twenty-one, and the largest consequences.
- **Terraform version**, pinned per workspace, so upgrades can be staged.
- **VCS connection** and branch, plus a **working directory** for monorepos.
- **Auto-apply**, which applies a successful plan with no approval. Rarely right for production.
- **Notifications**, and run tasks calling an external service during a run.

> ON THE EXAM
>
> The working directory setting is how a monorepo of configurations maps onto many workspaces from one repository.

Projects

- **Projects group workspaces**, so their access, variables and settings can be managed collectively.
- **Permissions apply at the project level**, so a team gets access to a group rather than to each workspace.
- A workspace belongs to **one project**, and can be moved between them in its settings.
- **Variable sets can be scoped to a project**, so every workspace in it inherits those values automatically.
- **Including future ones**, which is what makes self-service workflows possible.

> ON THE EXAM
>
> The 'and future ones' part is the point. A new workspace in the project inherits credentials and permissions without anyone configuring it.

Why that enables self-service

- Create a project for a team, and apply a **variable set with their cloud credentials** to it.
- Grant the team **permission to create workspaces** within that project.
- **Every workspace they create inherits the credentials and the permissions**, with no platform team involvement.
- HashiCorp names this directly: broader variable set scope **enables self-service workflows**.
- Without projects, each new workspace needs credentials and permissions configured by hand.

> ON THE EXAM
>
> This is objective 8c's real point. Projects are not just folders; they are the boundary that permissions and variables attach to.

Terraform variables or environment variables?

TERRAFORM VARIABLES

Inputs to the configuration

- Correspond to variable blocks in your code
- Referenced as var.name
- An HCL toggle allows complex types
- The equivalent of a .tfvars entry

ENVIRONMENT VARIABLES

Set in the run environment

- Exported into the shell running Terraform
- Used for provider credentials
- Also TF_LOG and other TF_ settings
- Not visible to var.name

THE TIEBREAKER

Both live in the same workspace screen and behave differently. Credentials belong in the environment category.

Sensitive variables

- Either kind can be marked **sensitive**, which is how credentials are stored in a workspace.
- **A sensitive value is write-only in the interface.** You can replace it; you cannot read it back.
- So the service holds the secret, rather than a file, a pipeline configuration, or somebody's laptop.
- **Dynamic credentials go further**, storing no long-lived secret at all. Module twenty-one.
- **The value still reaches state if a provider returns it.** Module four's rule has not changed.

> ON THE EXAM
>
> Storing a secret in a workspace solves distribution, not persistence. That distinction is module thirteen's, and it still applies here.

Variable sets

- **A reusable collection of Terraform and environment variables**, created once and applied to many places.
- Variables in a set **behave exactly like workspace variables**: available during runs, can be sensitive, support HCL types.
- The obvious use: **one set of cloud credentials**, applied wherever they are needed.
- Rotating a credential then means **editing one variable set**, not every workspace.
- They are created at the **organisation level**, then scoped to where they should apply.

> ON THE EXAM
>
> The value is in rotation and consistency. One set edited once, instead of the same secret pasted into forty workspaces.

The three variable set scopes

SCOPE	APPLIES TO
Global	All current and future workspaces in the organisation
Project-scoped	All current and future workspaces in the selected projects
Workspace-scoped	Only the workspaces you select

ON THE EXAM

Global and project scopes both include future workspaces. That is what makes them useful, and what makes them worth reviewing.

Precedence: the rules that matter

- **Workspace-specific variables always overwrite variables from variable sets** with the same key.
- **Workspace-scoped and project-scoped sets take precedence over global sets.**
- **Workspace-specific variables and workspace-scoped sets beat project-scoped sets.**
- The pattern: **the more specific the scope, the higher the precedence**.
- So a global default can be overridden for one workspace, which is the intended design.

> ON THE EXAM
>
> More specific wins. That is the opposite direction from module eight's file ordering, and the one exception is on the next slide.

When two sets of the same scope collide

- **Two variable sets at the same scope can define the same key.** Something has to break the tie.
- **HCP Terraform compares the variable set names and uses lexical order.**
- So a set named starting with A wins over one starting with B, for a conflicting key.
- **Not the most recently edited one.** Editing a set does not change which one wins.
- Ordering is by **Unicode code point**, so it is a genuine string comparison rather than anything cleverer.

> ON THE EXAM
>
> A conflict resolved by alphabetical order is invisible unless you know the rule. Prefer not creating the conflict.

Priority variable sets

- A variable set can be marked as a **priority** set, which inverts the usual direction.
- **Its values overwrite variables with the same key set at more specific scopes.**
- **Including command line flags, .auto.tfvars files and terraform.tfvars.**
- The use case: an organisation enforcing a value that individual workspaces must not override.
- **It does not guarantee the configuration uses the variable.** Somebody can hardcode a value instead.

> ON THE EXAM
>
> HashiCorp's own advice: for stricter enforcement use policy checks or run tasks. A priority variable set is not a control.

Run triggers

- **A run trigger links two workspaces**, so a successful apply in one queues a run in another.
- The use case: a networking workspace applies, and the workspaces that depend on it re-plan.
- Values usually flow between them through **remote state outputs**, as in module seven.
- **It queues a run; it does not apply one.** Approval still follows the destination workspace's settings.
- This is how a large estate is split into several workspaces without losing the ordering between them.

> ON THE EXAM
>
> Splitting one configuration into several workspaces was module five's advice about -target. Run triggers are what keeps them coordinated.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
One workspace should differ from the default	A workspace-specific variable. More specific wins
Rotate a credential used by forty workspaces	One variable set, edited once
Let a team create its own workspaces	A project, with a variable set scoped to it
Two sets define the same key	Lexical order of the set names decides
Engineers must not override this value	A priority variable set, with its caveat
Re-plan downstream when networking changes	A run trigger, plus remote state outputs

ON THE EXAM

Most questions here reduce to scope. Ask how many workspaces the value should reach, and whether it must be overridable.

> A platform team creates a global variable set defining instance_type as a small size, so every workspace gets a sensible default. Production workspaces need a larger size, so an engineer sets a workspace-specific variable with the same key on each of them.
>
> A colleague worries the global set will override the production values.

What actually happens?

A The global variable set wins, because it is defined at the organisation level

B The workspace-specific variable wins. Workspace variables always overwrite variable set values

C The run fails with a conflicting variable definition error

D Whichever was edited most recently wins

◆ PAUSE AND DECIDE

What actually happens?

A The global set wins

Backwards. Organisation level means broadest scope, and the broadest scope has the lowest precedence unless the set is marked as priority.

B The workspace variable wins

Correct. HashiCorp states workspace-specific variables always overwrite variables from variable sets with the same key.

C A conflict error

Overriding is the intended design, not an error. HCP Terraform shows which value it will use rather than refusing to run.

D Most recently edited

The common assumption, and wrong even where a tie-break is needed. Same-scope conflicts are broken by lexical order of set names.

THE SIGNAL

Set a default broadly, override it narrowly. That is what the precedence order is designed to allow.

> A new squad needs to manage its own infrastructure. They will create roughly one workspace per service, expected to reach thirty over a year. Each needs the same cloud credentials and the same team permissions.
>
> The platform team currently configures credentials and permissions on each new workspace by hand, which takes about a day per request.

What should the platform team set up?

A. A global variable set, so every workspace in the organisation gets the credentials

B. A project for the squad, a variable set scoped to it, and permission to create workspaces there

C. A workspace-scoped variable set listing all thirty workspaces in advance

D. A single shared workspace with CLI workspaces inside it for each service

◆ PAUSE AND DECIDE

What should the platform team set up?

A A global variable set

Would give one squad's credentials to every workspace in the organisation, including other teams'. Broad scope is the wrong tool for a bounded group.

B A project and a scoped set

Correct. Project scope covers current and future workspaces, so each new one inherits credentials and permissions with no ticket.

C List thirty in advance

They do not exist yet, and the list would need editing on every creation. It reintroduces the manual step it was meant to remove.

D CLI workspaces inside one

Module eighteen: a CLI workspace shares one configuration, one credential set and one permission boundary. It is not an environment or a service.

THE SIGNAL

'Current and future workspaces' is the phrase that decides this. Only global and project scopes include workspaces that do not exist yet.

> Two project-scoped variable sets are applied to the same project. One is named aws-shared and defines a region as one value. The other is named zz-overrides, created later specifically to change that region, and defines the same key with a different value.
>
> Runs continue to use the original region. The engineer who created zz-overrides edits it again, with no effect.

Why?

A Project-scoped sets cannot define the same key twice, so the second is ignored entirely

B Same-scope conflicts are resolved by lexical order of the set names, and aws-shared sorts first

C Variable sets are cached, and the project must be re-saved to pick up edits

D The newer set needs to be marked sensitive before it takes effect

◆ PAUSE AND DECIDE

Why?

A **Second set ignored**

It is not ignored; it is applied and then loses the tie. Its other, non-conflicting variables take effect normally.

B **Lexical order decides**

Correct. HCP Terraform compares variable set names and uses the one with lexical precedence. 'aws-shared' sorts before 'zz-overrides'.

C **Caching**

Invents a mechanism. Edits take effect on the next run; the edited value is simply losing the same tie every time.

D **Mark it sensitive**

Sensitivity controls whether a value can be read back in the interface. It has no bearing on precedence.

THE SIGNAL

Naming a set to sort last is exactly backwards. The correct fix is a workspace-specific variable, or a priority set.

> A compliance team must ensure every workspace tags resources with a cost centre from an approved list. Engineers currently pass their own value with -var, and some workspaces override the organisation default.
>
> The compliance team asks whether a priority variable set will guarantee the tag is correct everywhere.

What is the accurate answer?

A. Yes. A priority variable set overrides everything, including -var, so the value is guaranteed

B. It will override -var and tfvars files, but cannot stop somebody hardcoding a value in the configuration

C. No. Priority variable sets affect only environment variables, not Terraform variables

D. Yes, provided every workspace is moved into a single project first

◆ PAUSE AND DECIDE

What is the accurate answer?

A Guaranteed

Overstates it. HashiCorp says a priority set cannot guarantee the configuration uses the variable at all - only what the variable's value would be.

B Overrides inputs, not code

Correct, and it is HashiCorp's own caveat. For stricter enforcement they recommend policy checks or run tasks.

C Environment variables only

False. Variable sets hold both kinds, and priority applies to both.

D One project first

Scope affects reach, not the strength of the override. It also does not address the hardcoding gap.

THE SIGNAL

The gap is real: a priority set controls the variable's value, and a hardcoded literal never reads the variable. Policy is the control.

Module 22 recap

- **A workspace holds configuration, state, variables, run history and permissions.**
- **Projects group workspaces** so access, variables and settings are managed collectively.
- **Global and project scopes include future workspaces.** That is what enables self-service.
- **More specific scope wins**, and workspace variables always beat variable sets.
- **Same-scope conflicts are broken alphabetically**, not by which was edited last.
- **A priority set inverts the order**, and still cannot stop a hardcoded value.

> ON THE EXAM
>
> Run triggers queue a run in a downstream workspace after an upstream apply. They do not approve it.